
Experiment Plan for Two-Timescale CTR Forecasting

Long-term temporal adaptation plus within-day online calibration

Core idea. For each impression, first obtain a long-term CTR estimate from models trained on historical days. Then apply a lightweight scalar calibration correction that is updated causally from labels observed earlier in the current day.

$q_{d,i}$ = long-term prediction for impression i on day d ,

$\hat{p}_{d,i} = \text{sigmoid}(\text{logit}(q_{d,i}) + b_{d,i})$,

$b_{d,i+1} = \text{Proj}_{[-B,B]}[b_{d,i} - \eta_i(\hat{p}_{d,i} - y_{d,i})]$.

Primary question	Do long-term cross-day adaptation and short-term within-day calibration provide complementary improvements in CTR forecasting?
Primary metric	Chronological test log loss, with paired uncertainty estimates over days.
Main comparison	Combined method versus long-only and short-only ablations.
Evaluation principle	Every prediction and update must use only information available before that prediction.

1 Objective and Testable Hypotheses

The experiment should determine whether the two temporal components solve different problems:

- The **long-term module** adapts to changes across days by combining predictors trained on different historical horizons.
- The **short-term module** carries fresh information within the current day through an online calibration state.

For day d and impression i , the long-term module outputs $q_{d,i}$. The long-term predictor is frozen during day d . The online module maintains a scalar $b_{d,i}$ and predicts

$$\hat{p}_{d,i} = \text{sigmoid}(\text{logit}(\tilde{q}_{d,i}) + b_{d,i}), \quad \tilde{q}_{d,i} = \min\{1 - \varepsilon, \max\{\varepsilon, q_{d,i}\}\}, \quad (1)$$

where clipping by a small ε avoids unstable logits.

The main hypotheses are:

H1: Long-term value.

The adaptive long-term predictor improves over simple fixed historical windows.

H2: Short-term value.

Online calibration improves a fixed long-term predictor by using labels observed earlier in the current day.

H3: Complementarity.

The combined method improves over both the long-only and short-only variants.

H4: Drift sensitivity.

Calibration gains are larger on days or intraday intervals with larger frozen-model residual drift.

H5: Causal robustness.

Gains persist under realistic feedback delays and do not rely on access to future labels.

Primary estimand. The most important quantity is the paired test-day loss difference

$$\Delta_d = L_d(\text{Combined}) - L_d(\text{Long-only}).$$

A negative mean with a paired 95% confidence interval below zero supports the value of within-day calibration on top of long-term adaptation.

2 Proposed Method

2.1 Long-term cross-day prediction

Use the existing multi-timescale prediction bank. At the beginning of day d , construct candidate predictors using only days before d , for example

$$f_d^{(3)}, \quad f_d^{(7)}, \quad f_d^{(\text{exp})}, \quad (2)$$

corresponding to rolling 3-day, rolling 7-day, and expanding histories. The existing adaptive mechanism produces

$$q_{d,i} = \sum_{h \in \mathcal{W}} \alpha_{d,h} f_d^{(h)}(x_{d,i}), \quad \sum_{h \in \mathcal{W}} \alpha_{d,h} = 1, \quad (3)$$

with weights computed from information available before day d .

For a clean decomposition, all $f_d^{(h)}$ and $\alpha_{d,h}$ are held fixed throughout day d .

2.2 Within-day online calibration

Initialize

$$b_{d,1} = 0. \quad (4)$$

After a label becomes available, update the intercept using the logistic-loss gradient:

$$b_{d,i+1} = \text{Proj}_{[-B,B]} [b_{d,i} - \eta_i (\hat{p}_{d,i} - y_{d,i})]. \quad (5)$$

The state requires $O(1)$ memory and $O(1)$ work per update. A positive value of b raises all current predictions on the logit scale; a negative value lowers them.

For efficiency and realistic label maturity, the production-style implementation may update once per time block k rather than after every impression:

$$g_{d,k} = \frac{1}{|M_{d,k}|} \sum_{i \in M_{d,k}} (\hat{p}_{d,i} - y_{d,i}), \quad b_{d,k+1} = \text{Proj}_{[-B,B]} (b_{d,k} - \eta_k g_{d,k}), \quad (6)$$

where $M_{d,k}$ contains only labels that have matured by the update time.

2.3 Causal replay algorithm

For each evaluation day d :

1. Fit or load all long-term candidate models using labels from days $< d$.
2. Compute the long-term mixture and freeze it for day d .
3. Set $b = 0$ and place arriving impressions in timestamp order.
4. For each impression, predict using Equation (1) with the current b .
5. When earlier labels have matured, update b using Equation (5) or (6).
6. At the end of the day, record metrics and release day d for use in constructing day $d + 1$ models.

3 Chronological Evaluation Protocol

Use the existing 116-day chronological split:

Segment	Days	Purpose
Initial training	1–60	Fit initial base models and initialize historical experts.
Development	61–81	Select all hyperparameters and finalize the protocol.
Locked test	82–116	One-pass final evaluation; no tuning or method changes.

Within every test day, replay impressions in exact timestamp order. Let Δ denote the assumed feedback-maturation delay. A label from impression time τ can affect predictions only after $\tau + \Delta$.

Use

$$\Delta = 30 \text{ minutes} \quad (7)$$

as the main setting, with sensitivity checks at

$$\Delta \in \{0, 15, 60\} \text{ minutes.} \quad (8)$$

Because the dataset does not reveal the exact click-label arrival time, all conclusions should explicitly state the delay assumption.

3.1 Leakage controls

- Do not use day d labels to construct $q_{d,i}$ or the long-term mixture weights.
- Do not update b from an outcome until the chosen delay has elapsed.
- Select horizons, regularization, clipping, learning rate, delay, and update frequency only on development days.
- Use identical base predictions across methods whenever possible so that comparisons isolate the temporal mechanism.
- Use time-of-day only in the designated seasonality baseline, not silently in the proposed method.

4 Baseline and Ablation Suite

Method	Long-term component	Within-day component	Purpose
Expanding	All previous days	None	Stable conventional historical baseline.
Rolling-3	Previous 3 days	None	Fast fixed-window adaptation.
Rolling-7	Previous 7 days	None	Intermediate fixed-window adaptation.
Equal ensemble	Equal average of expanding, rolling-3, and rolling-7	None	Simple multi-timescale baseline without learned adaptation.
Long-only	Existing adaptive multi-timescale method	None	Isolates the value of cross-day adaptation.
Short-only	Expanding model	Online intercept	Isolates the value of within-day calibration.
Combined	Adaptive multi-timescale method	Online intercept	Proposed full method; tests complementarity.
Time-of-day	Expanding or long-only	Fixed hourly intercept learned from prior days	Tests whether online gains merely reflect repeating daily seasonality.
Oracle intercept	Any chosen long-term predictor	Best hindsight intercept for the full day	Non-deployable upper bound on scalar calibration value.

A useful optional stronger baseline is full online Platt scaling,

$$\hat{p}_{d,i} = \text{sigmoid}(a_{d,i} \text{logit}(q_{d,i}) + b_{d,i}), \quad (9)$$

which tests whether the intercept-only policy gives nearly all available benefit with lower complexity and greater stability.

The two decisive comparisons are

Combined vs. Long-only and Combined vs. Short-only.

The first isolates the value of fresh within-day information; the second determines whether adaptive cross-day learning remains useful once online calibration is present.

5 Feasibility Diagnostics

Before a full hyperparameter search, quantify how much scalar calibration can possibly help.

5.1 Daily oracle improvement

For each development day, solve

$$b_d^* = \arg \min_{b \in [-B, B]} \sum_i \ell(\text{sigmoid}(\text{logit}(q_{d,i}) + b), y_{d,i}). \quad (10)$$

Compare the uncalibrated and oracle losses. If the oracle improvement is negligible, an intercept-only online method has little room to help.

5.2 Intraday residual structure

For each 30-minute or hourly interval $I_{d,k}$, plot

$$\bar{r}_{d,k} = \frac{1}{|I_{d,k}|} \sum_{i \in I_{d,k}} (y_{d,i} - q_{d,i}). \quad (11)$$

Persistent positive or negative runs indicate intraday calibration drift. Also inspect whether the residual process differs across days and whether it is predictable from earlier same-day observations.

5.3 Early-day versus late-day behavior

Divide each day into equal-duration segments. A causal online calibrator should initially match the long-term model and improve only after current-day information accumulates. Large gains from the first segment would signal leakage, an initialization artifact, or a hidden time feature.

6 Hyperparameter Selection

Keep the search intentionally small. Tune only on development days.

Projection radius	$B \in \{0.25, 0.5, 1, 2\}$.
Learning rate	$\eta_k = \eta_0 / \sqrt{k}$ with $\eta_0 \in \{0.01, 0.03, 0.1, 0.3\}$, or a constant rate selected from the same scale.
Update frequency	Per impression, 15 minutes, 30 minutes, or 60 minutes.
Feedback delay	Main: 30 minutes. Sensitivity: 0, 15, and 60 minutes.
Base clipping	$\varepsilon \in \{10^{-6}, 10^{-5}, 10^{-4}\}$, primarily for numerical stability.
Daily initialization	Main: $b_{d,1} = 0$. Ablation: $b_{d,1} = \rho b_{d-1,\text{end}}$ with $\rho \in \{0.5, 1\}$.

Select one configuration by mean development log loss, with a stability tie-breaker favoring smaller B , fewer updates, and less variation in worst-day loss. Freeze this configuration before opening the locked test results.

7 Metrics and Statistical Analysis

7.1 Primary predictive metric

Report impression-weighted log loss:

$$L = \frac{1}{N} \sum_{d,i} [-y_{d,i} \log \hat{p}_{d,i} - (1 - y_{d,i}) \log(1 - \hat{p}_{d,i})]. \quad (12)$$

Also report the unweighted mean of daily log loss so that high-volume days do not completely determine the conclusion.

7.2 Robustness and calibration

Report:

- median and worst-day log loss;
- number and fraction of test days won against expanding and long-only;
- Brier score and expected calibration error;
- mean residual and log loss by intraday time block;
- early-day, middle-day, and late-day performance;
- paired 95% bootstrap confidence intervals over test days.

AUC is secondary because an additive intercept leaves ranking unchanged inside an update block. The principal value of the short-term module should appear in proper scoring rules and calibration.

7.3 Downstream autobidding evaluation

Run the existing matched-budget bidding evaluation with each method's pCTR estimates. Report value, conversions, spend, and pacing-relevant quantities under the same budget and auction stream. This tests whether better calibration translates into operational value rather than only lower log loss.

7.4 Computational cost

Measure:

- total fit and inference time;
- calibration-update time per million impressions;
- additional state and memory;
- number of model retrains and synchronization points.

The short-term intercept should add negligible cost relative to base-model scoring.

8 Theory-Aligned Evaluation

The online intercept can be compared with the best fixed intercept in hindsight for each day. Define

$$R_d = L_d^{\text{online}} - L_d^{\text{oracle}}, \quad (13)$$

where L_d^{oracle} uses b_d^* from Equation (10). Report both absolute regret and the fraction of available scalar-calibration improvement captured:

$$\text{CapturedGain}_d = \frac{L_d^{\text{long}} - L_d^{\text{online}}}{L_d^{\text{long}} - L_d^{\text{oracle}}}. \quad (14)$$

Exclude or separately mark days whose denominator is numerically negligible.

For logistic loss convex in b , projected online gradient descent admits a standard sublinear regret guarantee against the best fixed $b \in [-B, B]$. The empirical quantities above directly show whether the finite-sample behavior approaches that comparator.

9 Ablations and Sanity Checks

Run the following minimal checks:

1. **Daily reset versus carry-over.** Determine whether the short-term state should represent only the current day or persist across days.
2. **Immediate versus delayed feedback.** Quantify how much performance depends on unrealistically fast labels.
3. **Update granularity.** Compare per-impression and 15-, 30-, and 60-minute block updates.
4. **Fixed seasonality versus current-day evidence.** Compare online calibration with the time-of-day baseline.
5. **Chronology placebo.** Shuffle impression order within each day before applying the calibrator. A substantial reduction in gain supports the role of chronological information.
6. **Long-term redundancy.** Compare combined performance when the long-term module is expanding, equal ensemble, and adaptive multi-timescale.
7. **Calibration complexity.** Compare intercept-only calibration with full online Platt scaling if the latter is stable.

10 Success Criteria and Decision Rules

The combined method is empirically supported if the locked test shows all of the following:

1. It improves mean log loss over **Long-only**, with a paired 95% confidence interval for the daily loss difference below zero.
2. It improves over **Short-only**, showing that adaptive long-term learning is not redundant.
3. It outperforms the fixed time-of-day calibrator, indicating that current-day observations carry information beyond stable seasonality.
4. It remains beneficial under at least one nonzero feedback-delay setting.
5. It improves calibration and captures a nontrivial fraction of the daily oracle-intercept gain.
6. It does not obtain its average gain from a small number of extreme days while materially harming most stable days.
7. Its computational overhead remains negligible relative to training and scoring the base CTR models.

Interpretation of outcomes.

- If **Short-only** $\approx$ **Combined**, focus the paper on efficient causal online calibration; the adaptive long-term module adds little.
- If **Long-only** $\approx$ **Combined**, the dataset has limited exploitable within-day calibration drift or labels arrive too late.
- If **Combined beats both**, the two-timescale framework is supported and becomes the main algorithmic result.
- If only the oracle helps, improve the online update policy before increasing model complexity.

11 Recommended Execution Order

1. Generate frozen long-term predictions for all development and test impressions.
2. Run the oracle-intercept and intraday-residual diagnostics.
3. Implement the intercept-only chronological replay with delayed labels.
4. Compare expanding, long-only, short-only, and combined methods first.
5. Add rolling windows, equal ensemble, time-of-day, and optional full online Platt scaling.
6. Tune the small hyperparameter grid on development days.
7. Freeze code and configuration; run the locked 35-day test exactly once.
8. Produce paired statistical comparisons, calibration plots, delay sensitivity, and computational-cost results.
9. Run the matched-budget autobidding evaluation.
10. Replicate the fixed protocol on a second chronological CTR dataset if the main result is positive.